

Лабораторная работа № 6. Хранение и обработка данных в мобильных приложениях

Цели лабораторной работы: получить навыки работы с разными типами хранилищ, включая SharedPreferences, DataStore, базу данных на примере SQLite библиотеки для работы с ней;

Задачи лабораторной работы:

- изучить возможности по использованию SharedPreferences, DataStore;
- использовать абстрактные методы onCreate, OnUpgrade для работы с базой данных, выполнять выборки данных.

Table of Contents

Критерии оценивания.....	2
Содержание отчета.....	3
Настройка .gitignore для проекта в Android Studio.....	4
9.1. Использование SharedPreferences.....	5
Теоретические сведения.....	5
Записать в общие настройки.....	6
Чтение общих настроек.....	7
ПРИМЕР 1. Хранение параметров приложения в Shared Preferences.....	8
ЗАДАНИЕ 1. Хранение параметров приложения в Shared Preferences.....	11
9.2. Использование Android Jetpack Preferences DataStore.....	12
Теоретические сведения.....	12
Типы DataStore.....	12
ЗАДАНИЕ 2. Использование Android Jetpack Preferences DataStore.....	12
ЗАДАНИЕ 3. Использование Android Jetpack Proto DataStore.....	13
9.3. Работа с базой данных.....	13
Теоретические сведения.....	13
1. Работа с базой данных через raw SQL-запросы.....	13
2. Компонент Android для работы с данными Content Provider.....	16
3. Библиотека Room для работы с SQLite.....	16
ЗАДАНИЕ 4. Приложение для работы с базой данных SQLite с помощью raw-запросов.....	17
ЗАДАНИЕ 5. Android Content Provider для работы с базой данных SQLite.....	21
ЗАДАНИЕ 6. Android Room для работы с базой данных SQLite.....	21
ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ.....	21
ЗАДАНИЕ 7.1. Хранение параметров приложения в Shared preferences.....	21
ЗАДАНИЕ 7.2. Хранение параметров приложения в Preferences DataStore.....	22
ЗАДАНИЕ 7.3. Хранение параметров приложения в Proto DataStore.....	22
ЗАДАНИЕ 7.4. Приложение с хранением данных в БД SQLite с помощью raw запросов.....	23
ЗАДАНИЕ 7.5. Обновление схемы базы данных SQLite.....	23
ЗАДАНИЕ 7.6. Приложение с хранением данных в БД и возможностями фильтрации, сортировки, группировки и агрегации с Content Provider.....	24
ЗАДАНИЕ 7.7. Приложение с хранением данных в БД и возможностями фильтрации, сортировки, группировки и агрегации с Android Room.....	24
ЗАДАНИЕ 7.8. Приложение для подкастов с хранением настроек и данных в БД	

.....	24
Варианты.....	27
Вариант 1.....	27
Вариант 2.....	28
Вариант 3.....	28
Вариант 4.....	28
Вариант 5.....	29
Вариант 6.....	29
Вариант 7.....	29
Вариант 8.....	30
Вариант 9.....	30
Вариант 10.....	30
Вариант 11.....	31
Вариант 12.....	31
Вариант 13.....	31
Вариант 14.....	32
Вариант 15.....	32
Вариант 16.....	33
Вариант 17.....	33
Вариант 18.....	33
Вариант 19.....	34
Вариант 20.....	34
Вариант 21.....	34
Вариант 22.....	35
Вариант 23.....	35
Вариант 24.....	35
Вариант 25.....	36
Контрольные вопросы.....	36

Критерии оценивания

4-5 — приложения на основе примера 1, заданий 1-6, задание 7 для самостоятельной работы (7.1 на Java с Shared Preferences) из лабораторной работы и ответы на контрольные вопросы;

6-7 — приложения на основе примера 1, задания 1-6, задание 7 для самостоятельной работы (7.1, 7.2, 7.4, 7.5, 7.7 на языке Java или Kotlin) из лабораторной работы и ответы на контрольные вопросы;

8 — приложения на основе примера 1, задания 1-6, задание 7 для самостоятельной работы (7.1-7.6, 7.8 на языке Java или Kotlin) из лабораторной работы и ответы на контрольные вопросы. **Генерация кода на языке Kotlin из java-проекта не разрешается;**

9 — приложения на основе примера 1, задания 1-6, задание 7 для самостоятельной работы (7.1-7.8 на языке Java или Kotlin) из лабораторной работы, дополнительный функционал в заданиях для самостоятельной работы и ответы на контрольные вопросы. **Генерация кода на языке Kotlin из java-проекта не разрешается.**

Каждый проект опубликовать в отдельной папке репозитории. Работа с

ветками приветствуется.

Содержание отчета

1. Цель работы.
2. Скриншоты приложений.
3. Ответы на контрольные вопросы.

Отчет должен быть опубликован в git-репозитории на github. Все результаты лабораторной работы должны быть опубликованы в git-репозитории, ссылка на который доступна в курсе «Программирование мобильных и встраиваемых систем»:

В файле Readme проекта на github должна быть ссылка на отчёт. Отчет опубликовать во внешнем хранилище или в репозитории в каталоге /docs. Если в лабораторной работе необходимо написать программу/ы, то отчёт должен результаты тестов по каждой программе и ответы на контрольные вопросы.

Пример оформления файла Readme может быть таким:

```
# Overview

Report on LabRabota___.
// Указать название и номер лабораторной работы

# Projects

// Описать разработанные проекты.

## How to build

// Рекомендации по сборке проекта/ов, включая требования к версии Android ОС.

# Author

Your name and group number.

# Additional Notes
//дополнительные комментарии.
```

Каждая лабораторная работа содержит тексты задач и контрольные вопросы, ответы на которые проверяются преподавателем при приёме работы у студента.

Выполнение студентом лабораторной работы и сдача её результатов преподавателю происходит следующим образом:

1. Студент выполняет разработку программ.
2. В ходе разработки студент обязан следовать указаниям к данной задаче (в случае их наличия). Исходные тексты программ следует разрабатывать в соответствии с требованиями к оформлению, приведёнными в приложении.
3. Студент выполняет самостоятельную проверку исходного текста каждой разработанной программы и правильности её работы, а также свои знания по

теме лабораторной работы.

Настройка .gitignore для проекта в Android Studio

! Каждый проект в репозитории должен содержать файл .gitignore.

Пример .gitignore представлен ниже. Еще один пример .gitignore – [github Android.gitignore example](#).

```
# built application files
*.apk
*.ap_
*.aab

# Mac files
.DS_Store

# files for the dex VM
*.dex

# Java class files
*.class

# generated files
bin/
gen/

# Ignore gradle files
.gradle/
build/

# Local configuration file (sdk path, etc)
local.properties

# Proguard folder generated by Eclipse
proguard/
proguard-project.txt

# Eclipse files
.project
.classpath
.settings/

# Android Studio/IDEA
*.iml
.idea

# External native build folder generated in Android Studio 2.2 and later
.externalNativeBuild

# Google Services (e.g. APIs or Firebase)
google-services.json

# Android Studio captures folder
captures/

# Others
.cxx
```

9.1. Использование SharedPreferences

Теоретические сведения

SharedPreferences — постоянное хранилище на платформе Android, используемое приложениями для хранения своих настроек, например. Это хранилище является относительно постоянным, пользователь может зайти в настройки приложения и очистить данные приложения, тем самым очистив все данные в хранилище.

В Android **Shared Preferences** используются для сохранения и извлечения данных примитивных типов данных (целочисленные, плавающие, логические, строковые, длинные) в виде пары ключ-значение из файла в файловой структуре приложения.

Обычно объект **Shared Preferences** указывает на файл, который содержит пары ключ-значение и предоставляет простые методы чтения и записи для сохранения и извлечения пар ключ-значение из файла.

Файл общих настроек управляется платформой Android, и к нему можно получить доступ из любого места приложения для чтения или записи данных в файл, но получить доступ к файлу из любого другого приложения невозможно, поэтому он защищен.

Общие настройки полезны для хранения небольшой коллекции значений ключей, таких как информация для входа пользователя в систему, настройки приложения, связанные с пользователями и т. Д., Для поддержания состояния приложения в следующий раз, когда они снова войдут в приложение.

Для взаимодействия с данными применяется SharedPreference API, классы которого наследуются от Preference API. В 29-ой версии API Preference API был переведён в статус deprecated. На данный момент используется новая версия API — `androidx.preference:preference` (<https://developer.android.com/develop/ui/views/components/settings>).

Чтобы создать новый файл общих настроек или получить доступ к существующему, нужно вызвать один из двух методов: **getSharedPreferences()** или **getPreferences()**.

- **getSharedPreferences()** — Используйте этот метод, если вам потребуется несколько общих файлов настроек, идентифицируемых по имени, указанном в первом параметре. Для работы с данными постоянного хранилища нам понадобится экземпляр класса **SharedPreferences**, который можно получить у любого объекта, унаследованного от класса *android.content.Context* (например, **Activity** или **Service**). У объектов этих классов (унаследованных от **Context**) есть метод **getSharedPreferences**, который принимает 2 параметра:

- **name** — выбранный файл настроек. Если файл настроек с таким

именем не существует, он будет создан при вызове метода `edit()`.

- **mode** — режим работы. Возможные значения:
 - **MODE_PRIVATE** — используется в большинстве случаев для приватного доступа к данным приложением-владельцем
 - **MODE_WORLD_READABLE** — только для чтения
 - **MODE_WORLD_WRITEABLE** — только записи
 - **MODE_MULTI_PROCESS** — несколько процессов совместно используют один файл **SharedPreferences**.

- `getPreferences()` — Используйте метод из `Activity`, если вам нужен только один файл общих настроек для операции. Поскольку при этом получается файл общих настроек по умолчанию, относящийся к операции, вам не нужно указывать имя.

Если мы используем один общий файл настроек для нашей `Activity`, то нам необходимо инициализировать объект `SharedPreferences` с помощью метода `getPreferences()`:

```
SharedPreferences sharedPref = getPreferences(Context.MODE_PRIVATE);
```

Если у нас несколько файлов настроек, то инициализировать объект `SharedPreferences` можем следующим образом:

```
SharedPreferences sharedPref =  
getSharedPreferences("filename1", Context.MODE_PRIVATE);
```

`filename1` в данном случае — это файл настроек, из которого читаем значения. Режим доступа `MODE_PRIVATE`, что означает файл доступен только приложению, к которому относится.

Чтобы получить значение необходимой переменной, используйте следующие методы объекта **SharedPreferences**:

- **getBoolean**(String key, boolean defValue),
- **getFloat**(String key, float defValue),
- **getInt**(String key, int defValue),
- **getLong**(String key, long defValue),
- **getString**(String key, String defValue),
- **getStringSet**(String key, Set defValues).

Второй параметр — значение, которое вернется в случае если значение по ключу `key` отсутствует в **SharedPreferences**. Также, методом **getAll()** можно получить все доступные значения.

Записать в общие настройки

Для записи в файл общих настроек создайте объект `SharedPreferences.Editor` посредством вызова `edit()` в `SharedPreferences`.

Передавать ключи и значения, которые хотите записать, с помощью методов:

- **putBoolean**(String key, boolean value),
- **putFloat**(String key, float value),
- **putInt**(String key, int value),
- **putLong**(String key, long value),
- **putString**(String key, String value),
- **putStringSet**(String key, Set values)

Рассмотрим пример (см. код ниже). Чаще всего для передачи ключей и значений с помощью таких методов, как `putInt()` и `putString()`. Затем вызовите `commit()` для сохранения изменений. Например:

Листинг 1

```
SharedPreferences sharedPref = getPreferences(Context.MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPref.edit();
editor.putBoolean("keyname", true);
editor.putString("keyname", "string value");
editor.putInt("keyname", "int value");
editor.putFloat("keyname", "float value");
editor.putLong("keyname", "long value");
editor.commit();
```

Листинг 2

```
SharedPreferences sharedPref =
getActivity().getPreferences(Context.MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPref.edit();
editor.putInt(getString(R.string.saved_high_score), newHighScore);
editor.commit();
```

Чтение общих настроек

Для получения значений из файла общих настроек следует вызвать такие, методы как `getInt()` и `getString()`, предоставляя ключ для нужного вам значения, а также при желании значение по умолчанию, которое будет выводиться при отсутствии ключа. Например:

Листинг 3

```
SharedPreferences pref = getPreferences(Context.MODE_PRIVATE);
pref.getString("keyname", null);
pref.getInt("keyname", 0);
pref.getFloat("keyname", 0);
pref.getBoolean("keyname", true);
pref.getLong("keyname", 0);
```

Листинг 4

```
SharedPreferences sharedPref =
getActivity().getPreferences(Context.MODE_PRIVATE);
int defaultValue = getResources().getInteger(R.string.saved_high_score_default);
long highScore = sharedPref.getInt(getString(R.string.saved_high_score),
defaultValue);
```

Удаление данных из общих настроек

Мы можем удалить все данные из файла общих настроек, используя метод `clear()`, как показано ниже.

```
SharedPreferences pref = getPreferences(Context.MODE_PRIVATE);
SharedPreferences.Editor editor = pref.edit();
editor.clear();
editor.commit();
```

Используя фрагмент кода выше, мы удаляем все значения из общих настроек, используя метод `clear()`, и фиксируем изменения в файле общих настроек, используя метод `commit()`.

Теперь рассмотрим, как хранить и извлекать пары ключ-значение примитивного типа данных в файле общих настроек, используя объект `SharedPreferences` в приложении для Android.

ПРИМЕР 1. Хранение параметров приложения в Shared Preferences

Ниже приведен пример хранения и извлечения значений примитивных типов данных из файла общих настроек с использованием `SharedPreferences`.

Изучить документацию

<https://developer.android.com/develop/ui/views/components/settings>.

Создайте новое приложение из одной Activity в Android Studio и сохраните под именем **SharedPreferences Example**.

После того, как создадите приложение, откройте файл **activity_main.xml** в папке **res/layout** и напишите код, как показано ниже:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <EditText
        android:id="@+id/txtName"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="50dp"
        android:ems="10" />

    <EditText
        android:id="@+id/txtPwd"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="50dp"
        android:ems="10"
        android:inputType="textPassword" />

    <Button
        android:id="@+id/btnLogin"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginLeft="100dp"
        android:text="Login" />

    <TextView
        android:id="@+id/fstTxt"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```



```

        android:layout_marginLeft="100dp"
        android:layout_marginTop="150dp"
        android:text="UserName" />

<TextView
    android:id="@+id/secTxt"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginLeft="100dp"
    android:text="Password" />
</LinearLayout>

```

Создайте другой файл разметки с именем **details.xml** в папке **res/layout**., щелкнув правой кнопкой мыши по папке **layout** и выбрав их контекстного меню **New** команду **Layout Resource File**.

Пример кода для файла **details.xml**:

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:id="@+id/resultView"
        android:layout_gravity="center"
        android:layout_marginTop="170dp"
        android:textSize="20dp"/>
    <Button
        android:id="@+id/btnLogOut"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:layout_marginTop="20dp"
        android:text="Log Out" />
</LinearLayout>

```

Теперь откройте файл **MainActivity.java** и добавьте код ниже:

```

package com.example.myapplication;

import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;

import static android.content.Context.MODE_PRIVATE;

public class MainActivity extends AppCompatActivity {
    EditText uname, pwd;
    Button loginBtn;
    SharedPreferences pref;
    Intent intent;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        uname = (EditText)findViewById(R.id.txtName);
        pwd = (EditText)findViewById(R.id.txtPwd);
    }
}

```

```

loginBtn = (Button)findViewById(R.id.btnLogin);
pref = getSharedPreferences("user_details",MODE_PRIVATE);
intent = new Intent(MainActivity.this,DetailsActivity.class);
if(pref.contains("username") && pref.contains("password")){
    startActivity(intent);
}
loginBtn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        String username = uname.getText().toString();
        String password = pwd.getText().toString();
        if(username.equals("fpmi-student") && password.equals("student")){
            SharedPreferences.Editor editor = pref.edit();
            editor.putString("username",username);
            editor.putString("password",password);
            editor.commit();
            Toast.makeText(getApplicationContext(), "Login
Successful",Toast.LENGTH_SHORT).show();
            startActivity(intent);
        }
        else
        {
            Toast.makeText(getApplicationContext(),"Credentials are not
valid",Toast.LENGTH_SHORT).show();
        }
    }
});
}
}

```

В данном коде проверяем, соответствуют ли введенные данные имени пользователя и пароля или нет. Если введены верно, то сохраняем данные в файл общих настроек и перенаправляем пользователя на другую Activity. Обратите внимание, что в коде задано имя пользователя **fpmi-student** и пароль **student**.

Создайте новый файл **DetailsActivity.java**, который будет выводить данные из общих настроек. Код файла ниже:

```

package com.example.myapplication;
import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;

import androidx.appcompat.app.AppCompatActivity;

import static android.content.Context.MODE_PRIVATE;

public class DetailsActivity extends AppCompatActivity {
    SharedPreferences prf;
    Intent intent;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.details);
        TextView result = (TextView)findViewById(R.id.resultView);
        Button btnLogOut = (Button)findViewById(R.id.btnLogOut);
        prf = getSharedPreferences("user_details",MODE_PRIVATE);
        intent = new Intent(DetailsActivity.this,MainActivity.class);
        result.setText("Hello, "+prf.getString("username",null));
        btnLogOut.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {

```

```

        SharedPreferences.Editor editor = prf.edit();
        editor.clear();
        editor.commit();
        startActivity(intent);
    }
});
}
}

```

Добавим описание созданной Activity в файл AndroidManifest.xml:

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapplication">

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <activity android:name=".DetailsActivity" android:label="Shared Preferences -
Details"></activity>
    </application>

</manifest>

```

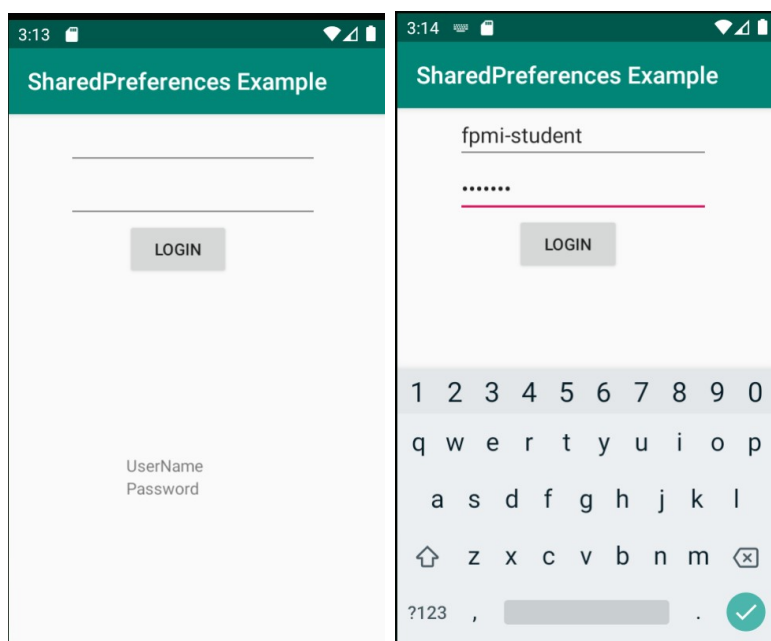
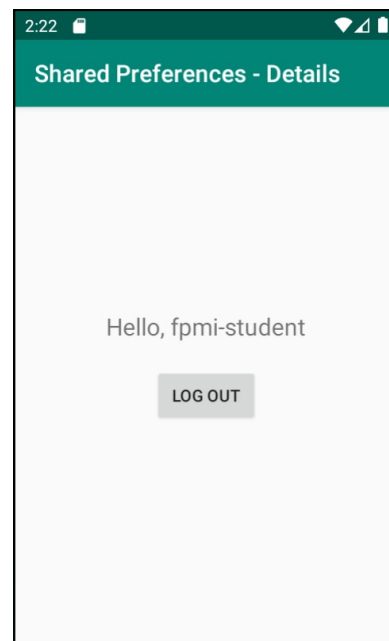


Рис. 1. а) Макет Activity.xml



б) Макет details.xml

ЗАДАНИЕ 1. Хранение параметров приложения в Shared Preferences

Реализовать приложение из примера 1 на языке Java, заменив **LinearLayout** на **ConstraintLayout**, добавив поля в файл **strings.xml** и внеся изменения в код приложения, например вывод даты последнего входа из файла настроек.

9.2. Использование Android Jetpack Preferences DataStore

Теоретические сведения

Jetpack DataStore — это библиотека для хранения данных в Android, которая заменяет устаревший SharedPreferences. Jetpack DataStore в отличие от SharedPreferences работает асинхронно. Вся работа с библиотекой выполняется с помощью Kotlin Coroutines и Flow. DataStore позволяет нам хранить данные двумя способами:

- По принципу «ключ — значение» аналогично SharedPreferences.
- Хранить структурированные данные на основе Protocol Buffers.

Типы DataStore

- **Preferences DataStore** — хранит данные по принципу «ключ — значение». Подходит для хранения небольших наборов данных в виде пар «ключ-значение», аналогично SharedPreferences, например для пользовательских настроек (тема, язык, флаги состояний) и других данных, которые не требуют сложной структуры. Позволяет мигрировать с SharedPreferences с минимальными изменениями кода.

- **Proto DataStore** — используется для хранения структурированных данных с заранее определённой схемой. Подходит для сложных объектов, перечислений, списков и других структурированных данных. Обеспечивает типобезопасность — данные проверяются на соответствие схеме на этапе компиляции, что снижает риск ошибок во время выполнения. Подходит для сценариев, где важна эффективность сериализации (например, хранение статей с заголовками, контентом и временными метками). Также поддерживает версионирование данных, что упрощает обработку изменений структуры данных со временем.

Подробнее:

Базовое руководство по DataStore, включая описание обоих вариантов реализации <https://developer.android.com/topic/libraries/architecture/datastore>

<https://habr.com/ru/company/tinkoff/blog/525010/>

<https://android-developers.googleblog.com/2020/09/prefer-storing-data-with-jetpack.html>

<https://www.raywenderlich.com/18348259-datastore-tutorial-for-android-getting-started>

ЗАДАНИЕ 2. Использование Android Jetpack Preferences DataStore

Изучить статьи из раздела 9.2. Использование Android Jetpack Preferences DataStore, <https://habr.com/ru/articles/874034/> и пример реализации <https://c1ctech.com/android-jetpack-preferences-datastore-example/>. Реализовать

приложение из примера, рассмотренного в последней статье.

ЗАДАНИЕ 3. Использование Android Jetpack Proto DataStore

Изучить статьи из раздела 9.2. Использование Android Jetpack Preferences DataStore и пример <https://c1ctech.com/android-jetpack-proto-datastore-example/>. Реализовать приложение из примера, рассмотренного в статье.

9.3. Работа с базой данных

Теоретические сведения

SQLite

- SQLite—компактная встраиваемая реляционная база данных с открытым исходным кодом. Поддерживает динамическое типизирование данных.
- Возможные типы полей: INTEGER, REAL, TEXT, BLOB.

В приложении, при подключении к БД мы указываем **имя БД** и **версию**. При этом могут возникнуть следующие ситуации:

1) БД **не существует**. Это может быть, например, в случае первичной установки программы. В этом случае приложение должно само **создать** БД и все таблицы в ней. И далее оно уже работает с только что созданной БД.

2) БД **существует**, но ее версия **устарела**. Это может быть в случае обновления программы. Новой версии программы нужны дополнительные поля в старых таблицах или новые таблицы. В этом случае приложение должно обновлять существующие таблицы и создать новые, если это необходимо.

3) БД **существует** и ее версия **актуальна**. В этом случае приложение успешно **подключается** к БД и работает.

Для работы с базой данных в проектах для Android применяются следующие способы:

1. Работа с базой данных через raw SQL-запросы.
2. Компонент Android для работы с данными Content Provider
3. Библиотека Room для работы с SQLite

1. Работа с базой данных через raw SQL-запросы

Этот метод предполагает прямое взаимодействие с SQLite через стандартные классы Android. Можно использовать в проектах на Java и Kotlin.

Основные классы:

- SQLiteOpenHelper — базовый класс для управления созданием и обновлением базы данных.
- SQLiteDatabase — класс для выполнения SQL-запросов (insert, update, delete, query).
- Cursor — интерфейс для чтения результатов запросов.

Пример:

```
SQLiteDatabase db = dbHelper.getWritableDatabase();
```

```

db.execSQL("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY,
name TEXT)");
db.insert("users", null, values);
Cursor cursor = db.query("users", null, null, null, null, null, null);

```

Для обработки нам надо создать **класс**, являющийся наследником для [SQLiteOpenHelper](#). Назовем его DBHelper. Этот класс предоставит нам методы для **создания** или **обновления** БД в случаях ее **отсутствия** или **устаревания**:

SQLiteOpenHelper

Абстрактные методы

onCreate – метод, который будет вызван, если БД, к которой мы хотим подключиться – не существует

onUpgrade – будет вызван в случае, если мы пытаемся подключиться к БД более новой версии, чем существующая

Чтение из базы данных

Пример чтения данных из базы данных:

```

SQLiteDatabase db = dbHelper.getReadableDatabase();

// Define a projection that specifies which columns from the database
// you will actually use after this query.
String[] projection = {
    BaseColumns._ID,
    FeedEntry.COLUMN_NAME_TITLE,
    FeedEntry.COLUMN_NAME_SUBTITLE
};

// Filter results WHERE "title" = 'My Title'
String selection = FeedEntry.COLUMN_NAME_TITLE + " = ?";
String[] selectionArgs = { "My Title" };

// How you want the results sorted in the resulting Cursor
String sortOrder =
    FeedEntry.COLUMN_NAME_SUBTITLE + " DESC";

Cursor cursor = db.query(
    FeedEntry.TABLE_NAME,    // The table to query
    projection,              // The array of columns to return (pass null to get
all)                        // all)
    selection,               // The columns for the WHERE clause
    selectionArgs,           // The values for the WHERE clause
    null,                   // don't group the rows
    null,                   // don't filter by row groups
    sortOrder               // The sort order
);

```

Позиционирование курсора

- `moveToFirst()` - перемещает курсор в первую запись в выборке;

- moveToLast() - перемещает курсор в последнюю запись в выборке;
- moveToNext() - перемещает курсор в следующую запись и одновременно определяет, существует ли эта запись. Метод moveToNext() возвращает true, если курсор указывает на другую строку после перемещения, и false, если текущая запись была последней в выборке;
- moveToPrevious() - перемещает курсор в предыдущую запись;
- moveToPosition() - перемещает курсор в указанную позицию;
- getPosition() - возвращает текущий индекс позиции курсора.
- isFirst();
- isLast();
- isBeforeFirst();
- isAfterLast().

Обновление и удаление записей

Обновление записей:

```

SQLiteDatabase db = dbHelper.getWritableDatabase();

// New value for one column
String title = "MyNewTitle";
ContentValues values = new ContentValues();
values.put(FeedEntry.COLUMN_NAME_TITLE, title);

// Which row to update, based on the title
String selection = FeedEntry.COLUMN_NAME_TITLE + " LIKE ?";
String[] selectionArgs = { "MyOldTitle" };

int count = db.update(
    FeedEntry.TABLE_NAME,
    values,
    selection,
    selectionArgs);

```

Удаление записей:

```

// Define 'where' part of query.
String selection = FeedEntry.COLUMN_NAME_TITLE + " LIKE ?";
// Specify arguments in placeholder order.
String[] selectionArgs = { "MyTitle" };
// Issue SQL statement.
int deletedRows = db.delete(FeedEntry.TABLE_NAME, selection, selectionArgs);

```

Документация:

- [Официальная документация SQLite](#)
- [SQLiteDatabase](#)
- [SQLiteOpenHelper](#)

2. Компонент Android для работы с данными Content Provider

Content Provider — это компонент Android, который позволяет приложениям обмениваться данными через стандартный интерфейс. Он часто используется для доступа к данным, хранящимся в базе данных, но может работать и с другими источниками (например, файлами).

Основные классы:

ContentProvider — базовый класс для создания провайдера.

ContentResolver — класс для взаимодействия с провайдерами (например, query, insert, update, delete).

UriMatcher — помогает определить, к какой таблице или ресурсу относится запрос.

ContentUris — утилиты для работы с URI.

Пример:

```
// В провайдере:
public class MyContentProvider extends ContentProvider {
    @Override
    public Cursor query(Uri uri, String[] projection, String selection,
String[] selectionArgs, String sortOrder) {
        // Логика запроса к базе данных
    }
}

// В клиентском коде:
Cursor cursor = getContentResolver().query(MyContentProvider.CONTENT_URI,
null, null, null, null);
```

Документация и статьи:

- [Content Provider](#)
- [ContentResolver](#)
- [Создание Content Provider](#)

3. Библиотека Room для работы с SQLite

Room — это библиотека, которая упрощает работу с SQLite, предоставляя ORM и проверку запросов на этапе компиляции.

Основные классы:

- @Entity — аннотация для сущностей (таблиц).
- @Dao — аннотация для интерфейсов доступа к данным.
- @Database — аннотация для класса базы данных.
- RoomDatabase — базовый класс для работы с Room.

Пример на Java:

```
@Entity(tableName = "users")
public class User {
```



```

    @PrimaryKey(autoGenerate = true)
    public int id;
    public String name;
}

@Dao
public interface UserDao {
    @Query("SELECT * FROM users")
    List<User> getAllUsers();

    @Insert
    void insert(User user);
}

@Database(entities = {User.class}, version = 1)
public abstract class AppDatabase extends RoomDatabase {
    public abstract UserDao userDao();
}

// Использование:
AppDatabase db = Room.databaseBuilder(context, AppDatabase.class, "my-
db").build();
db.userDao().insert(new User());

```

Подробнее в [официальной документации Room](#).

ЗАДАНИЕ 4. Приложение для работы с базой данных SQLite с помощью raw-запросов

Создадим простое приложение – справочник контактов, которое будет хранить **имя** и **email**. **Вводить** данные будем на **экране** приложения, а для **отображения** информации используем **журнал** (логи). Обычно для вывода используется List (список).

Открываем **MainActivity.java** и пишем:

```

public class MainActivity extends Activity implements OnClickListener {

    final String LOG_TAG = "myLogs";

    Button btnAdd, btnRead, btnClear;
    EditText etName, etEmail;

    DBHelper dbHelper;

    /** Called when the activity is first created. */
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        btnAdd = (Button) findViewById(R.id.btnAdd);
        btnAdd.setOnClickListener(this);

        btnRead = (Button) findViewById(R.id.btnRead);
        btnRead.setOnClickListener(this);
    }
}

```

```

        btnClear = (Button) findViewById(R.id.btnClear);
        btnClear.setOnClickListener(this);

        etName = (EditText) findViewById(R.id.etName);
        etEmail = (EditText) findViewById(R.id.etEmail);

        // создаем объект для создания и управления версиями БД
        dbHelper = new DBHelper(this);
    }

    @Override
    public void onClick(View v) {

        // создаем объект для данных
        ContentValues cv = new ContentValues();

        // получаем данные из полей ввода
        String name = etName.getText().toString();
        String email = etEmail.getText().toString();

        // подключаемся к БД
        SQLiteDatabase db = dbHelper.getWritableDatabase();

        switch (v.getId()) {
        case R.id.btnAdd:
            Log.d(LOG_TAG, "--- Insert in mytable: ---");
            // подготовим данные для вставки в виде пар: наименование столбца -
            значение

            cv.put("name", name);
            cv.put("email", email);
            // вставляем запись и получаем ее ID
            long rowID = db.insert("mytable", null, cv);
            Log.d(LOG_TAG, "row inserted, ID = " + rowID);
            break;
        case R.id.btnRead:
            Log.d(LOG_TAG, "--- Rows in mytable: ---");
            // делаем запрос всех данных из таблицы mytable, получаем Cursor
            Cursor c = db.query("mytable", null, null, null, null, null, null);

            // ставим позицию курсора на первую строку выборки
            // если в выборке нет строк, вернется false
            if (c.moveToFirst()) {

                // определяем номера столбцов по имени в выборке
                int idColIndex = c.getColumnIndex("id");
                int nameColIndex = c.getColumnIndex("name");
                int emailColIndex = c.getColumnIndex("email");

                do {

```

```

        // получаем значения по номерам столбцов и пишем все в лог
        Log.d(LOG_TAG,
            "ID = " + c.getInt(idColIndex) +
            ", name = " + c.getString(nameColIndex) +
            ", email = " + c.getString(emailColIndex));
        // переход на следующую строку
        // а если следующей нет (текущая - последняя), то false -
выходим из цикла
    } while (c.moveToNext());
} else
    Log.d(LOG_TAG, "0 rows");
c.close();
break;
case R.id.btnClear:
    Log.d(LOG_TAG, "--- Clear mytable: ---");
    // удаляем все записи
    int clearCount = db.delete("mytable", null, null);
    Log.d(LOG_TAG, "deleted rows count = " + clearCount);
    break;
}
// закрываем подключение к БД
dbHelper.close();
}

class DBHelper extends SQLiteOpenHelper {

    public DBHelper(Context context) {
        // конструктор суперкласса
        super(context, "myDB", null, 1);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        Log.d(LOG_TAG, "--- onCreate database ---");
        // создаем таблицу с полями
        db.execSQL("create table mytable ("
            + "id integer primary key autoincrement,"
            + "name text,"
            + "email text" + ");");
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int
newVersion) {
    }
}
}

```

В методе Activity - **onCreate** мы определяем объекты, присваиваем обработчики и создаем объект dbHelper класса **DBHelper** для управления БД. Сам

класс будет описан ниже.

Далее смотрим метод Activity – **onClick**, в котором обрабатываем нажатия на кнопки.

Класс [ContentValues](#) используется для указания **полей** таблицы и **значений**, которые в эти поля будем вставлять. Создаем объект **cv**, и позже его используем. Далее записываем в переменные значения из **полей** ввода. Затем, с помощью метода [getWritableDatabase](#) подключаемся к БД и получаем объект [SQLiteDatabase](#). Он позволит нам работать с БД. Будем использовать его методы **insert** – вставка записи, **query** – чтение, **delete** – удаление.

Далее смотрим, какая кнопка была нажата:

btnAdd – добавление записи в таблицу *mytable*. Заполняем объект **cv** парами: **имя** поля и **значение**. И (при вставке записи в таблицу) в указанные поля будут вставлены соответствующие значения. Заполняем поля *name* и *email*. *id* у нас заполнится автоматически (primary key autoincrement). Вызываем метод [insert](#) – передаем ему **имя таблицы** и объект **cv** с вставляемыми значениями. Второй аргумент метода используется, при вставке в таблицу пустой строки. Нам это сейчас не нужно, поэтому передаем **null**. Метод **insert** возвращает **ID** вставленной строки, мы его сохраняем в **rowID** и выводим в **лог**.

btnRead – чтение всех записей из таблицы *mytable*. Для чтения используется метод [query](#). На вход ему подается **имя таблицы**, список запрашиваемых полей, условия выборки, группировка, сортировка. Т.к. нам нужны все данные во всех полях без сортировок и группировок - мы используем везде **null**. Только имя таблицы указываем. Метод возвращает нам объект класса [Cursor](#). Его можно рассматривать как таблицу с данными. Метод [moveToFirst](#) – делает **первую** запись в **Cursor** **активной** и заодно проверяет, есть ли вообще записи в нем (т.е. выдалось ли что-либо в методе **query**). Далее мы получаем порядковые номера столбцов в **Cursor** по их именам с помощью метода [getColumnIndex](#). Эти номера потом используем для чтения данных в методах [getInt](#) и [getString](#) и выводим данные в **лог**. С помощью метода [moveToNext](#) мы перебираем все строки в **Cursor** пока не добиремся до последней. Если же записей не было, то выводим в **лог** соответствующее сообщение – *0 rows*. В конце закрываем курсор (освобождаем занимаемые им ресурсы) методом [close](#), т.к. далее мы его нигде не используем.

btnClear – очистка таблицы. Метод [delete](#) удаляет записи. На вход передаем **имя таблицы** и **null** в качестве условий для удаления, а значит удалится все. Метод возвращает **кол-во удаленных** записей.

После этого закрываем соединение с БД методом [close](#).

Класс **DBHelper** является **вложенным** в **MainActivity** и описан в конце кода. Этот класс должен наследовать класс **SQLiteOpenHelper**.

В **конструкторе** мы вызываем конструктор суперкласса и передаем ему:

context - контекст

mydb - название базы данных

null – объект для работы с курсорами, нам пока не нужен, поэтому **null**

1 – версия базы данных

В методе **onCreate** этого класса мы используем метод `execSQL` объекта `SQLiteDatabase` для выполнения SQL-запроса, который создает таблицу. Этот метод вызывается, если БД не существует и ее надо создавать. По запросу видно, что мы создаем таблицу *mytable* с полями *id*, *name* и *email*.

Метод **onUpgrade** пока не заполняем, т.к. используем одну версию БД и менять ее не планируем.

Важно понять, что для работы с БД мы использовали два класса:

- **DBHelper**, наследующий **SQLiteOpenHelper**. В его конструкторе мы вызываем конструктор супер-класса и указываем имя и версию БД. Метод **getWritableDatabase** выполняет подключение к базе данных и возвращает нам объект **SQLiteDatabase** для работы с ней. Метод **close** закрывает подключение к БД. В случае, когда БД отсутствует или устарела, класс предоставляет нам самим реализовать создание или обновление в методах **onCreate** и **onUpgrade**.

- **SQLiteDatabase**. Содержит методы для работы с данными – т.е. **вставка, обновление, удаление и чтение**.

ЗАДАНИЕ 5. Android Content Provider для работы с базой данных SQLite

Изучить статьи из раздела 9.3. Работа с базой данных и пример <https://c1ctech.com/android-content-provider-example-using-sqlite-database/>.

Реализовать приложение из примера, рассмотренного в статье.

ЗАДАНИЕ 6. Android Room для работы с базой данных SQLite

Изучить статьи из раздела 9.3. Работа с базой данных и пример на языке Java <https://c1ctech.com/android-room-persistence-library-example/> или Kotlin <https://c1ctech.com/android-room-database-with-mvvm-architecture/>. Реализовать любое из приложений из примера, рассмотренного в статье.

ЗАДАНИЯ ДЛЯ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

ЗАДАНИЕ 7.1. Хранение параметров приложения в Shared preferences

Изучить статью «Android: SharedPreferences (<http://developer.alexanderklimov.ru/android/theory/sharedpreferences.php>)», материалы и рекомендации из раздела 9.1. Использование SharedPreferences, примеры для Kotlin и Java из документации <https://developer.android.com/training/data-storage/shared-preferences> и изменить приложение «Угадай число», разработанное в лабораторных работах 2-5.

Продемонстрировать хранение настроек в **двух файлах настроек**. Сохранить в **один** файл настроек, например `app_settings`, используя для хранения перечисленных ниже параметров SharedPreferences:

- счетчик — количество запусков приложения;
- уникальный ID приложения, генерируемый при первом запуске;
- настройки пользователя (количество попыток, набранных баллов, лучший результат и др.);

Изучить пример <https://abhiandroid.com/programming/shared-preference>, добавить Activity авторизации (ввода адреса электронной почты и пароля) и сохранить во **второй** файл настроек, например `user_profile`, данные для авторизации пользователя:

- адрес электронной почты и пароль.

Приложение реализовать или на Java, или на Kotlin в зависимости от реализации приложения в лабораторных работах 2-5.

ЗАДАНИЕ 7.2. Хранение параметров приложения в Preferences DataStore

Изучить материалы и рекомендации из раздела 9.2. Использование Android Jetpack Preferences DataStore, примеры для Kotlin и Java из документации <https://developer.android.com/topic/libraries/architecture/datastore>, задание 2 текущей лабораторной работы и изменить приложение «Угадай число», разработанное в задании 7.1, заменив `SharedPreferences` на `Preferences DataStore`. Допускается применение миграции `SharedPreferences` в `Preferences DataStore`.

Продемонстрировать хранение настроек в **двух файлах настроек**. Сохранить в **один** файл настроек, например `app_settings`, используя для хранения перечисленных ниже параметров `Preferences DataStore`:

- счетчик — количество запусков приложения;
- уникальный ID приложения, генерируемый при первом запуске;
- настройки пользователя (количество попыток, набранных баллов, лучший результат и др.);

Данные для авторизации пользователя с Activity авторизации (ввода адреса электронной почты и пароля) сохранить во **второй** файл настроек, например `user_profile`:

- адрес электронной почты и пароль.

Приложение реализовать на Java или на Kotlin в зависимости от реализации приложения в задании 6.1.

ЗАДАНИЕ 7.3. Хранение параметров приложения в Proto DataStore

Изучить материалы и рекомендации из раздела 9.2. Использование Android Jetpack Preferences DataStore, примеры для Kotlin и Java из документации <https://developer.android.com/topic/libraries/architecture/datastore>, задание 2 текущей

лабораторной работы и изменить приложение «Угадай число», разработанное в заданиях 7.1 и 7.2.

Продemonстрировать хранение настроек в **двух файлах настроек**. Сохранить в **один** файл настроек, например `settings.pb`, используя для хранения перечисленных ниже параметров Proto DataStore:

- счетчик — количество запусков приложения;
- уникальный ID приложения, генерируемый при первом запуске;
- настройки пользователя (количество попыток, набранных баллов, лучший результат и др.);

Данные для авторизации пользователя с Activity авторизации (ввода адреса электронной почты и пароля) сохранить во **второй** файл настроек, например `user_profile.pb`:

- адрес электронной почты и пароль.

Приложение реализовать на Java или на Kotlin в зависимости от реализации приложения в заданиях 7.1/7.2.

ЗАДАНИЕ 7.4. Приложение с хранением данных в БД SQLite с помощью raw запросов

Изучить материалы раздела 9.3. Работа с базой данных, выполнить задание 4 и создать приложение, взаимодействующее с базой данных. Язык программирования Java или Kotlin. Первая операция (activity) должна содержать три кнопки. При нажатии на первую кнопку должна открываться новая операция (activity), выводящая информацию из таблицы согласно **варианту задания** (см. Варианты) в удобном для вывода формате.

При запуске приложения необходимо:

1. Создать БД, если ее не существует.
2. Создать таблицу согласно варианту.
3. Удалить все записи из БД (если существует), а затем добавить 5 записей.

При нажатии на вторую кнопку необходимо внести еще одну запись в таблицу.

При нажатии на третью кнопку необходимо заменить данные в любой записи.

ЗАДАНИЕ 7.5. Обновление схемы базы данных SQLite

Изучить материалы раздела 9.3. Работа с базой данных, выполнить задание 4 и создать новое отдельное приложение на основе приложения, созданного в упражнении 7.4, и дополнить схему базы данных согласно варианту (см. Варианты).

Переопределить функцию для обновления схемы базы данных. При изменении версии БД можно удалить таблицу с данными, сохранив данные, или добавить

столбцы с помощью ALTER. Обновление схемы базы данных должно включать два дополнительных поля следующих форматов: integer, char.

Обязательным условием при изменении структуры таблицы (базы данных) является изменение версии базы данных и сохранение данных, т. е. основная цель данного задания — проверить как правильно выполнять обновление приложения, если изменяется структура базы данных.

ЗАДАНИЕ 7.6. Приложение с хранением данных в БД и возможностями фильтрации, сортировки, группировки и агрегации с Content Provider

Изучить материалы раздела 9.3. Работа с базой данных, выполнить задание 5 и создать приложение на основе Content Provider на Java и/или Kotlin согласно варианту (см. Варианты). Реализовать в приложении следующие функции:

- вывод всех записей.
- вывод значения агрегирующей функции (SUM, MIN, MAX, COUNT, AVG)
- вывод строк большего определённого значения, которое задаётся или в текстовом поле, или списком выбора или другим способом.
- группировка строк по определённому признаку (признак может выбираться из списка).
- сортировка по различным полям.

ЗАДАНИЕ 7.7. Приложение с хранением данных в БД и возможностями фильтрации, сортировки, группировки и агрегации с Android Room

Изучить материалы раздела 9.3. Работа с базой данных, выполнить задание 6 и изменить приложение из задания 7.6, реализовав с использованием механизмов Android Room следующие функции:

- вывод всех записей.
- вывод значения агрегирующей функции (SUM, MIN, MAX, COUNT, AVG)
- вывод строк большего определённого значения, которое задаётся или в текстовом поле, или списком выбора или другим способом.
- группировка строк по определённому признаку (признак может выбираться из списка).
- сортировка по различным полям.

ЗАДАНИЕ 7.8. Приложение для подкастов с хранением настроек и данных в БД

Изменить приложение подкастов из задания 3 лабораторной работы 3, чтобы продемонстрировать использование трёх механизмов хранения данных Android: Preferences DataStore (для простых настроек), Proto DataStore (для

структурированных объектов) и Android Room (для реляционных данных). В задании ниже сформулированы минимальные требования, которые могут быть дополнены по желанию.

Приложение подкастов может **хранить такие типы данных** как:

- 1) Данные об артистах:
 - a) имя артиста;
 - b) ID артиста в сервисе;
 - c) URL изображения (аватар);
 - d) краткое описание (биография);
 - e) количество слушателей (популярность).
- 2) Данные о композициях (для каждого артиста — топ-10):
 - a) название трека;
 - b) длительность;
 - c) номер в чарте популярности;
 - d) ссылка на аудио;
 - e) год выпуска;
 - f) альбом (опционально).
- 3) Вспомогательные данные:
 - a) дата последнего обновления кеша для каждого артиста;
 - b) история поиска пользователя (последние 5–10 запросов артистов).

Минимальные требования к функциональности приложения:

- отображать список подкастов (название, автор, описание, дата выпуска);
- позволять пользователю отмечать подкасты как «прослушанные»;
- сохранять настройки приложения (язык интерфейса, размер шрифта);
- кешировать детали подкаста для офлайн-просмотра;
- показывать статистику прослушиваний.

Распределение по механизмам хранения:

- в Preferences DataStore хранить настройки приложения, например язык локализации, размер шрифта, тёмная тема, дата последнего входа в приложение и другие настройки по усмотрению;
- в Proto DataStore хранить кеш деталей подкаста, например id подкаста, название эпизода, описание, длительность, URL аудиофайла, дата кеширования;
- в Android Room хранить основные данные приложения, например база данных должна включать таблицы Podcast, Episode, PlaybackStats и другие таблицы при необходимости;

Технические требования:

1. Для настроек приложения реализовать класс AppPreferencesDataStore и методы для чтения и записи каждого параметра настроек.
2. Для хранения кеша деталей подкаста создать .proto файл со схемой данных, создать класс и реализовать методы для просмотра деталей, чтения и

очистки кеша.

3. Для хранения основных данных приложения создать сущности, создать DAO-интерфейсы с методами для CRUD-операций подкастов и эпизодов, обновления статуса прослушивания, получения статистики прослушиваний.

Пользовательский интерфейс включает:

1. главный экран со списком подкастов и фильтрацией по статусу прослушивания,
2. экран настроек (использует Preferences DataStore),
3. экран эпизода с информацией об эпизоде и кнопками «Отметить как прослушанное» и «Кешировать оффлайн» (использует Proto DataStore),
4. экран статистики с отображением времени прослушивания под подкастам (использует данные из Room).

Реализовать тесты для проверки логики и пользовательского интерфейса:

- 1 *Модульные тесты* для проверки логики работы отдельных компонентов:
 - 1.1 Preferences DataStore: корректность сохранения и загрузки настроек, значения по умолчанию, обработка ошибок, например при записи.
 - 1.2 Proto DataStore: сериализация/десериализация объектов, сохранение и чтение данных, очистка кеша.
 - 1.3 Android Room: DAO-методы (CRUD, запросы), конвертеры типов, миграции базы данных.
- 2 *Интеграционные тесты* для проверки взаимодействия между компонентами, например ViewModel + DataStore/Room, синхронизации данных между хранилищами, например при удалении подкаста из Room очистить кеш в Proto DataStore, обновления UI при изменении данных и др.
- 3 *Функциональные тесты* с тестированием полного цикла работы с подкастом, например поиск → кеширование → прослушивание → статистика, обработки при ошибках при отсутствии сети, недостаточно памяти, миграции данных при обновлении версии приложения. При реализации использовать Espresso для сценариев внутри приложения, UI Automator — для взаимодействия с системой, например сценарий использования инструментов тестирования может быть таким:
 - 3.1 Ввести название подкаста в поисковую строку (Espresso).
 - 3.2 Нажать «Кешировать» (Espresso).
 - 3.3 Проверить, что в уведомлениях появилось сообщение «Подкаст кеширован» (UI Automator).
 - 3.4 Перейти в офлайн-режим (UI Automator: настройки сети).
 - 3.5 Открыть детали подкаста — убедиться, что данные загружены из кеша (Espresso).
- 4 *Тесты UI* с Espresso и UI Automator для проверки корректности отображения интерфейса и реакции на действия пользователя, например отображение списка подкастов, навигация между экранами, обновление UI при изменении настроек (например, изменение шрифта), обработка состояний (загрузка,

ошибка, пустой список) и др.

Требования к поведению приложения:

1. При запуске загружать настройки из Preferences DataStore.
2. При просмотре деталей подкаста предлагать кеширование (Proto DataStore).
3. При прослушивании эпизода обновлять позицию воспроизведения в Room.
4. Показывать статистику на основе данных из Room.
5. Обеспечить синхронизацию между хранилищами: если подкаст удалён из Room, очистить его кеш в Proto DataStore.

Дополнительные рекомендации:

1. использовать Kotlin Flow для реактивного получения данных;
2. применять ViewModel для управления жизненным циклом данных;
3. обеспечить обработку ошибок (например, недоступности диска);
4. добавить журналирование (логирование) операций с хранилищами для отладки;
5. предусмотреть миграцию данных при обновлении схемы (для Room и Proto DataStore).

При оценивании выполнения задания будут учитываться следующие критерии:

1. все три механизма хранения данных реализованы и интегрированы;
2. данные корректно сохраняются и загружаются;
3. UI обновляется при изменении данных;
4. нет утечек памяти или блокировок UI-потока;
5. код соответствует принципам Clean Architecture;
6. добавлены модульные тесты для ключевых компонентов DataStore и Room DAO и другие виды тестов, критерии успешности которых следующие:
 - Все модульные и интеграционные тесты проходят (100 % coverage для критических путей).
 - Функциональные сценарии выполняются без ошибок в 95 % запусков.
 - UI-тесты стабильны (не падают из-за таймаутов или race conditions).
 - Отчёт о тестировании содержит детализацию по каждому типу тестов (количество пройденных/упавших).

Варианты

Варианты для заданий 7.4, 7.5 и 7.6.

Вариант 1.

7.4. Создать таблицу «Участники конференции», содержащую поля:

- ID;
- ФИО;
- Время регистрации на конференции.

7.5. Добавить поля «Компания», «Должность», «Адрес электронной почты».

7.6. База данных «Справочник Минска» состоит как минимум из 2 таблиц. Для районов Минска найти данные по микрорайонам количеству жителей, предприятий, школ, вузов, бюджету района.

Вариант 2.

7.4. Создать таблицу «Студенты», содержащую поля:

- ID;
- Фамилия;
- Имя;
- Дата зачисления.

7.5. Добавить поля «Место рождения», «Дата рождения», «Номер телефона».

7.6. База данных «Специальности ФПМИ» состоит как минимум из 2 таблиц. Хранить данные о курсах (1, 2, 3, 4), специальностях (ПМ, ПИ, ИН, КБ, АМ), предметах, читаемых на специальностях, и количестве студентов на каждой специальности каждого курса.

Вариант 3.

7.4. Создать таблицу «Книги», содержащую поля:

- ID;
- Автор;
- Название;
- Год выпуска.

7.5. Добавить поля «Город (где издана книга)», «Издательство», «Формат издания» (печатная или электронная книга).

7.6. База данных «Вузы Минска» состоит как минимум из 2 таблиц. Для вузов, хранить данные о названии, количеству факультетов, количеству специальностей, специальности и коды специальностей, количество студентов на специальности, а так же в каком вузе и на каком факультете обучают такой специальности, например студенты на специальности «Прикладная информатика» обучаются на факультетах ФПМИ, факультете социальнокультурных коммуникаций, факультете радиофизики и электроники в БГУ.

Вариант 4.

7.4. Создать таблицу «Специальность», содержащую поля:

- ID;
- Название специальности;
- Год открытия специальности.

7.5. Добавить поля «Курс обучения», «Количество студентов», «Специализация».

7.6. База данных «Издательство» состоит как минимум из 2 таблиц. Содержит сведения об авторах (фамилия, имя, отчество, емейл), гонорары (месяц, размер гонорара выплаченного автору, id книги, за которую выплачен гонорар), книги (название, автора, год издания, количество страниц) и др. данные.

Вариант 5.

7.4. Создать таблицу «Машины», содержащую поля:

- ID;
- Модель;
- Производитель;
- Год выпуска.

7.5. Добавить поля «Объем двигателя», «Количество лошадиных», «Тип кузова».

7.6. База данных «Магазин» состоит как минимум из 2 таблиц. Содержит данные о товарах, продавцах, стоимости товара, количестве проданных единиц, день продажи каждого товара, выручке для каждого продавца за каждый день.

Вариант 6.

7.4. Создать таблицу «Экзамен», содержащую поля:

- ID;
- Название предмета (дисциплины);
- Дата проведения экзамена

7.5. Добавить поля «Преподаватель», «Время начала экзамена», «Группа».

7.6. База данных «Страны» состоит как минимум из 2 таблиц. Содержит данные об общей площади, количестве населения, столице, городах (не менее 3), достопримечательностях и в каком городе находится достопримечательность.

Вариант 7.

7.4. Создать таблицу «Контакты», содержащую поля:

- ID;
- Фамилия;
- Имя
- Дата добавления записи.

7.5. Добавить поля «Отчество», «Адрес электронной почты», «Номер телефона».

7.6. База данных «Интернет магазин» состоит как минимум из 2 таблиц. Содержит сведения о товарах, стоимости товара, признаке наличия товара, заказах (номер, заказанные товары, способ оплаты), адресе доставки, курьере, статусе доставки.

Вариант 8.

7.4. Создать таблицу «Расписание», содержащую поля:

- ID;
- Название предмета;
- Время начала лекции/занятия.

7.5. Добавить поля «Время окончания занятия», «ФИО преподавателя», «Аудитория».

7.6. База данных «Сотрудники» состоит как минимум из 2 таблиц. Содержит сведения о личных данных сотрудников (фio, дата рождения, место рождения, номер телефона, электронной почты), паспортах, местах работы, занимаемых должностях, трудовом стаже и средней заработной плате по каждому месту работы.

Вариант 9.

7.4. Создать таблицу «Ноутбуки», содержащую поля:

- ID;
- Модель;
- Производитель.

7.5. Добавить поля «Время выпуска», «Процессор», «Объем оперативной памяти».

7.6. База данных «Студенты» состоит как минимум из 2 таблиц. Содержит сведения о личных данных студента (фio, дата рождения, место рождения, номер телефона, электронной почты), паспортные данные, результаты на момент поступления, результаты сдачи по сессиям (как минимум 2 сессии и 4 предмета по каждой сессии)

Вариант 10.

7.4. Создать таблицу «Музеи Минска», содержащую поля:

- ID;
- Название музея;
- Дата основания.

7.5. Добавить поля «Адрес», «Площадь музея», «Выставка (название выставки)».

7.6. База данных «Ресторан» состоит как минимум из 2 таблиц. Содержит

сведения о блюдах, их стоимости, времени на готовку блюда, включено или нет в обеденное меню, заказы, включая заказы с доставкой и данные о доставке, включая стоимость заказа, адрес доставки, способ оплаты и курьер и способ доставки.

Вариант 11.

7.4. Создать таблицу «Товар», содержащую поля:

- ID;
- наименование;
- стоимость.

7.5. Добавить поля «срок хранения», «сорт», «дата выпуска», «срок годности».

7.6. База данных «Музеи и библиотеки Минска» состоит как минимум из 2 таблиц. Для районов Минска найти данные по количеству музеев, библиотек, суммарное количество посетителей для музеев и библиотек по году, среднее количество посетителей по музеям и по библиотекам, максимальное и минимальное количество посетителей и др. данные.

Вариант 12.

7.4. Создать таблицу «Автомобиль», содержащую поля:

- ID;
- Марка;
- Серийный номер;
- Год выпуска.

7.5. Добавить поля «Цвет», «Год техосмотра», «Цена».

7.6. База данных «Кафедры» состоит как минимум из 2 таблиц. Для кафедр хранить сведения о заведующем, преподавателях, читаемых дисциплинах, количестве часов (лекционных и лабораторных). Учесть, что один преподаватель может читать несколько дисциплин и несколько преподавателей могут читать одну дисциплину. Вывести данные о суммарном количестве читаемых часов на преподавателя, с делением на лекционные и лабораторные, среднее количество часов на преподавателя по кафедре, минимальное и максимальное количество часов на преподавателя.

Вариант 13.

7.4. Создать таблицу «Государства», содержащую поля:

- ID;
- Название страны;
- Столица;

- Государственный язык.

7.5. Добавить поля «Население», «Площадь территории», «Денежная единица».

7.6. База данных «Учебники» состоит как минимум из 2 таблиц. Содержит сведения названия учебника, авторы, тираж, год издания, стоимость, язык учебника, класс обучения, издательство. Найти данные по количеству учебников для отдельного автора, суммарный тираж по одному учебнику в зависимости от языка, года издания, минимальный и максимальный тираж.

Вариант 14.

7.4. Создать таблицу «Абитуриент», содержащую поля:

- ID;
- Фамилия;
- Имя;
- Отчество;
- Дата рождения (год, месяц число).

7.5. Добавить поля «Гражданство», «Год поступления», «ЦТ (количество баллов)», «Аттестат (количество баллов)».

7.6. База данных «Издательство» состоит как минимум из 2 таблиц. Содержит сведения об видах печатных изделий (название, формат, тип бумаги, стоимость), заказы (дата заказа, количество экземпляров, количество страниц, стоимость, формат печати (одно/двухсторонняя)) и другие данные.

Вариант 15.

7.4. Создать таблицу «Рабочий», содержащую поля:

- ID;
- ФИО;
- Домашний адрес;
- Табельный номер;
- Должность.

7.5. Добавить поля «Образование», «Год поступления на работу», «Дата рождения (год, месяц число)».

7.6. База данных «БГУ» состоит как минимум из 2 таблиц. Для факультетов БГУ найти данные по количеству студентов, количеству специальностей, количеству студентов на бюджетной и платной форме обучения, количеству студентов по курсам (году обучения), средний, максимальный и минимальный балл студентов.

Вариант 16.

7.4. Создать таблицу «Владелец телефона», содержащую поля:

- ID;
- Фамилия;
- Имя;
- Отчество;
- Дата рождения;
- Номер телефона.

7.5. Добавить поля «Адрес», «Год заключения договора», «Название тарифа», «Стоимость тарифа».

7.6. База данных «Зоомагазин» состоит как минимум из 2 таблиц. Выводить список кормов с сортировкой, максимальную и минимальную стоимость одного корма в зависимости от производителя, среднюю стоимость одного корма, суммарное количество кормов.

Вариант 17.

7.4. Создать таблицу «Военнослужащий», содержащую поля:

- ID;
- ФИО;
- Домашний адрес (почтовый индекс, страна, область, район, город, улица, дом, квартира).

7.5. Добавить поля «Дата рождения», «Должность», «Звание», «Зарплата».

7.6. База данных «Пиццерия» состоит как минимум из 2 таблиц. Содержит сведения о видах блюд, их стоимости, акции, заказы, включая заказы с доставкой и данные о доставке, включая стоимость заказа и участие в акции, адрес доставки, способ оплаты, курьер и способ доставки.

Вариант 18.

7.4. Создать таблицу «Владелец автомобиля», содержащую поля:

- ID;
- ФИО;
- Номер телефона;
- Номер автомобиля.

7.5. Добавить поля «Марка автомобиля», «Номер техпаспорта», «Стоимость», «Год выпуска».

7.6. База данных «Аптека» состоит как минимум из 2 таблиц. Выводить

количество лекарств в каждой аптеке, среднюю стоимость одного лекарства по аптекам, минимальную и максимальную стоимость одного лекарства по аптекам.

Вариант 19.

7.4. Создать таблицу «Пациент», содержащую поля:

- ID;
- ФИО;
- Рост;
- Вес;
- Номер медицинской карты.

7.5. Добавить поля «Диагноз», «Группа крови», «Домашний адрес», «Номер телефона».

7.6. База данных «Онлайн-курса» состоит как минимум из 2 таблиц. Для каждого курса выводить количество студентов на каждый поток, среднее количество студентов по каждому курсу, максимальное и минимальное количество студентов, количество обученных студентов по каждому курсу и по году обучения.

Вариант 20.

7.4. Создать таблицу «Покупатель», содержащую поля:

- ID;
- ФИО;
- Номер телефона;
- Номер скидочной карты.

7.5. Добавить поля «Дата рождения», «Домашний адрес», «Адрес электронной почты», «Средний чек».

7.6. База данных «Аудиотека» состоит как минимум из 2 таблиц. Добавить функционал, обеспечивающий вывод среднего и суммарного количества песен в альбомах, максимальное количество песен и минимальное количество песен в одном альбоме, количество альбомов по году выпуска.

Вариант 21.

7.4. Создать таблицу «Студент», содержащую поля:

- ID;
- ФИО;
- Год поступления;
- Специальность;
- Факультет.

7.5. Добавить поля «Группа», «Курс», «Средний балл».

7.6. База данных «Фильмотека» состоит как минимум из 2 таблиц. Найти данные по количеству сеансов, среднему и суммарному количеству зрителей, максимальному и минимальному количеству зрителей за один сеанс.

Вариант 22.

7.4. Создать таблицу «Школьник», содержащую поля:

- ID;
- ФИО;
- Дата рождения;
- Школа;
- Класс.

7.5. Добавить поля «Классный руководитель», «Средний балл», «Адрес электронной почты».

7.6. База данных «Библиотека» состоит как минимум из 2 таблиц. Для авторов и тем найти данные по количеству книг автора/ темы, суммарной и средней стоимости, среднему тираже, количеству проданных книг и полученному доходу.

Вариант 23.

7.4. Создать таблицу «Студенты колледжа», содержащую поля:

- ID;
- Фамилия;
- Имя;
- Дата зачисления.
- Программа, на которую зачислен.
- Длительность обучения.

7.5. Добавить поля «Место рождения», «Дата рождения», «Дата окончания», «Специальность», «Квалификация». В поля «Специальность», «Квалификация» добавить данные только, если студент окончил колледж.

7.6. База данных «Факультеты БГУ» состоит как минимум из 2 таблиц. Хранить данные о факультетах(как минимум 3 факультета), курсах (1, 2, 3, 4), специальностях, количестве студентов на каждой специальности каждого курса, количестве преподавателей на каждой должности с учетом ставки и на каждом факультете.

Вариант 24.

7.4. Создать таблицу «Сотрудники», содержащую поля:

- ID;
- Фамилия;
- Имя;
- Дата принятия на работу.

7.5. Добавить поля «Идентификационный номер», «Дата рождения», «Номер телефона», «Должность», «Дата увольнения».

7.6. База данных «Товары и заказы» состоит как минимум из 2 таблиц. Хранить данные о компании (производителе), моделях микроволновых печей, дате выпуска, стоимости, НДС, количестве суммарно, на каком складе или в каком магазине находятся и другие данные.

Вариант 25.

7.4. Создать таблицу «Книги», содержащую поля:

- ID;
- Автор (ФИО);
- Название книги;
- Год издания.

7.5. Добавить поля «Издательство», «Количество страниц», «Стоимость».

7.6. База данных «Программное обеспечение» состоит как минимум из 2 таблиц. Содержит сведения о видах ПО, их стоимости, лицензии, акции, заказы, включая стоимость заказа, количество и типы лицензий, участие в акции, способ оплаты и др. данные.

Контрольные вопросы

1. Какие типы данных в можем хранить в Shared Preferences?
2. Какой метод используется в случае одного файла общих настроек в случае Shared Preferences?
3. Какой метод используется для нескольких файлов настроек в случае Shared Preferences?
4. Какие режимы работы (доступа) задаются при использовании SharedPreferences и какой из них самый надежный?
5. Приведите фрагменты кода для записи, чтения данных из файла общих настроек SharedPreferences и удаления данных.
6. Дайте характеристику методу хранения данных Preferences DataStore, его типам и в каких случаях его рекомендуется использовать.
7. Приведите фрагменты кода для записи, чтения данных из файла Preferences

DataStore в случае хранения данных по принципу «ключ — значение» по аналогии с SharedPreferences.

8. Опишите полный цикл миграции с SharedPreferences на Preference DataStore с использованием SharedPreferencesMigration. Укажите все потенциальные проблемы (конфликты ключей, типы данных, обработка ошибок) и способы их решения. Приведите пример кода с обработкой исключений и журналированием процесса миграции.

9. В чем принципиальное отличие механизма обеспечения потокобезопасности и согласованности данных (consistency) в Preference DataStore по сравнению с SharedPreferences? Опишите, как DataStore использует CoroutineContext и принцип «однократной записи» (single-writer principle) для предотвращения состояния гонки. Какие проблемы могут возникнуть при некорректной миграции с SharedPreferences на DataStore и как их диагностировать?

10. Разработайте архитектуру для приложения с несколькими изолированными профилями пользователей, где каждый профиль имеет собственные настройки, хранящиеся в Preference DataStore. Опишите, как вы организуете отдельные файлы DataStore для каждого профиля, обеспечите изоляцию данных и реализуете переключение между профилями без перезапуска приложения. Приведите примеры кода.

11. Объясните, как обеспечить транзакционную целостность при обновлении нескольких связанных настроек в Preference DataStore (например, тема интерфейса и цветовая схема). Опишите возможные проблемы при асинхронных операциях и предложите решение с использованием атомарных операций.

12. Использование Proto DataStore вместо Preference DataStore подразумевает работу с бинарным форматом Protocol Buffers. Какие преимущества в производительности (скорость парсинга, размер файла) и безопасности типов в этом случае получаем? Объясните, как поддерживать обратную совместимость схемы данных при добавлении новых полей в message (например, используя правила нумерации полей и ключевое слово reserved), чтобы приложение с новой версией не ломало данные, сохраненные старой версией, и наоборот.

13. Спроектируйте систему хранения сложных пользовательских настроек в Proto DataStore, включающую вложенные структуры (например, настройки уведомлений по категориям, расписание «не беспокоить», предпочтения контента). Опишите .proto-файл, сериализатор и механизм обновления отдельных полей без перезаписи всего объекта.

14. Какой класс используется для создания базы данных SQLite и управления версиями в случае взаимодействия с базой данных с помощью raw-запросов? Какой метод, который будет вызван, если БД, к которой подключаемся, не существует? Приведите его реализацию. Какой метод, будет вызван в случае, если подключаемся к БД более новой версии, чем существующая?

15. При использовании SQLiteOpenHelper с прямыми rawQuery и execSQL возникает проблема управления пулом соединений и многопоточности. Почему использование одного экземпляра SQLiteDatabase с флагом enableWriteAheadLogging(true) требует особой осторожности при параллельной записи и чтении? Объясните механизм исключений android.database.sqlite.SQLiteDatabaseLockedException и стратегию корректной обработки транзакций в многопоточной среде без использования ORM.

16. Напишите raw-запрос SQLite для получения топ-10 артистов с наибольшим количеством треков, у которых средняя длительность трека превышает 4 минуты. Включите JOIN с таблицей альбомов и фильтрацию по году выпуска. Объясните план выполнения запроса (EXPLAIN QUERY PLAN) и предложите индексы для оптимизации.

17. Реализуйте механизм пакетной вставки 10 000 записей в SQLite через raw-запросы с максимальной производительностью. Опишите использование транзакций, INSERT OR IGNORE, подготовленных выражений и буферизации. Сравните время выполнения с одиночными вставками.

18. Content Provider является абстракцией поверх БД, но его ключевая особенность — унификация доступа к данным через IPC (Binder). Объясните, как реализация query() в Content Provider должна обрабатывать параметр CancellationSignal и android:grantUriPermissions для обеспечения безопасности при передаче URI между приложениями. Какие риски существуют при использовании SQLiteDatabase.rawQuery внутри ContentProvider.query, если не учитывать лимиты Binder transaction buffer (1 Мб)?

19. Content Provider предоставляет метод applyBatch (пакетная операция). Опишите сценарий, в котором использование applyBatch внутри транзакции SQLite критически важно для сохранения целостности данных при синхронизации с сервером (например, в SyncAdapter). Как обеспечить атомарность операций вставки, обновления и удаления, поступающих из пакета, и почему стандартная реализация ContentProvider.applyBatch через цикл по ContentProviderOperation может быть небезопасна без явного управления транзакцией?

20. Спроектируйте Content Provider для обмена данными между двумя приложениями: одно записывает логи активности, другое отображает статистику. Опишите URI-схему, разрешения (read/write), реализацию query(), insert() и механизм уведомления об изменениях (ContentResolver.notifyChange()).

21. Объясните, как обеспечить безопасность Content Provider при предоставлении доступа к чувствительной информации (например, контакты). Опишите использование signature-разрешений, динамической проверки прав в onCall(), и механизма временных разрешений (grantUriPermission()). Приведите пример.

22. В чем заключается принципиальная разница между @Relation и @Embedded в Room при моделировании связей "один-ко-многим"? Объясните, как

Room генерирует код для методов, возвращающих `Map<Parent, List<Child>>`, и почему использование вложенных запросов (subqueries) в `@Query` с JOIN может быть более производительным, чем использование `@Relation` с дополнительными запросами (N+1 problem). Приведите пример, когда `@Relation` приводит к проблеме N+1 и как это исправить с помощью пользовательского JOIN запроса и `@Embedded/@ColumnInfo`.

23. Разработка приложения с Room требует управления миграциями базы данных. Опишите стратегию безопасной миграции, когда необходимо не только добавить новую таблицу, но и преобразовать данные существующей колонки (например, разделить поле `full_name` на `first_name` и `last_name`). Почему использование Migration с ALTER TABLE недостаточно, и как `Room.databaseBuilder(...).fallbackToDestructiveMigration()` может быть опасно в production? Как написать тест для проверки миграции с версии 1 на версию 2?

24. Room является оберткой над SQLite. В каких сценариях использование "сырых" raw-запросов через `SupportSQLiteDatabase` внутри Room (или вне его) может дать значительный прирост производительности по сравнению со стандартными подходами Room (`@Query`, `LiveData/Flow`)? Объясните влияние рефлексии и генерации кода на скорость холодного старта приложения и какие практики (например, использование `@RawQuery`) существуют для балансировки между безопасностью типов и производительностью сложных динамических запросов.

25. В архитектуре приложения часто используется Room для хранения основной предметной области (heavy data) и Proto DataStore для хранения пользовательских настроек (light state). Разработайте стратегию обеспечения согласованности данных между этими двумя источниками, когда настройка пользователя (например, "сортировка по умолчанию" в DataStore) влияет на запрос к базе данных (Room), а изменение данных в БД (например, удаление избранного элемента) должно обновлять состояние, зависящее от настроек. Как использовать Flow (из DataStore) и Flow/LiveData (из Room) для создания реактивной цепочки зависимостей без ручного вызова `invalidate`?